03 / EXERCISE GUIDE

EX30 — 3D rotations and matrices

Class 18 · Mathematics and engineering application

Goal and scope
Construct $\mathbf{R}_x(\theta)$, $\mathbf{R}_y(\theta)$ and $\mathbf{R}_z(\theta)$ from sine and cosine; apply them to vectors, investigate why 3D rotation order matters, and solve simple force and coordinate-change problems. One Python file only: no JSON, classes or `pytest`.

1. Exercise location

```
Python_Engineering/
02_numerical_methods/
algebra/
class_18_matrices/
EX28_matrix_operations.py
EX29_rotation_2D.py
EX30_rotation_3D.py  ← NEW
docs/
ES/
EN/
```

Working file: `EX30_rotation_3D.py`. Library: `import numpy as np`.

2. Model assumptions and notation

Right-handed 3D Cartesian system with fixed global x, y, z axes; positive angles follow the right-hand rule. Mathematical vectors are written as columns but represented by NumPy arrays of shape `(3,)`. Matrices have shape `(3,3)`.

Units: Part A uses unitless vectors; Parts B and C use forces in newtons. The lever arm is in metres and torque in N m.

3. First concept to implement: matrices $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$

Before rotating a vector, code the construction of **three different matrices**, each for a chosen θ:

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix} \qquad c = \cos\theta,\ s = \sin\theta$$

$$\mathbf{R}_y(\theta) = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}$$

$$\mathbf{R}_z(\theta) = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

1. Choose an angle in degrees and convert it to radians with `np.deg2rad()`.
2. Compute sine and cosine of that radian angle.
3. Construct the selected matrix as a 3×3 array, preserving signs and placement.
4. When the angle or axis changes, **rebuild** the matrix before using it again.

Do not enter precomputed values $0, 1, -1$ as a general solution. Calculate trigonometric entries from the input angle. A custom Python function is not needed yet.

4. Part A0 — Guided algebra example (verification)

Data: $\boldsymbol{a} = (2, 0, 1)^T$ and an active $+90°$ rotation about z.

1. Build $\mathbf{R}_z(90°)$ from sine and cosine.
2. Multiply by $\boldsymbol{a}$ and check by hand:

$$\mathbf{R}_z(90°) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{R}_z(90°)\boldsymbol{a} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}.$$

3. Confirm that both vector norms equal $\sqrt{5} \simeq 2.2361$.
4. Recover the original vector using $\mathbf{R}_z(90°)^T$ and compare arrays with `np.allclose()`.

5. Part A1 — Elementary rotations about X, Y and Z

Data: three **independent** cases: $\boldsymbol{b} = (1, 2, 0)^T$, $\boldsymbol{c} = (1, 0, 2)^T$ and $\boldsymbol{d} = (0, 3, 1)^T$.

1. Build $\mathbf{R}_x(+90°)$ and calculate $\boldsymbol{b}' = \mathbf{R}_x\boldsymbol{b}$.
2. Build $\mathbf{R}_y(+90°)$ and calculate $\boldsymbol{c}' = \mathbf{R}_y\boldsymbol{c}$.
3. Build $\mathbf{R}_z(-90°)$ and calculate $\boldsymbol{d}' = \mathbf{R}_z\boldsymbol{d}$.
4. Compare norms before and after each rotation.
5. For all three matrices, check $\mathbf{R}^T\mathbf{R} \simeq \mathbf{I}_3$ with `np.eye(3)` and `np.allclose()`. Check $\det \mathbf{R} \simeq 1$.

Expected: $\boldsymbol{b}' = (1, 0, 2)^T$, $\boldsymbol{c}' = (2, 0, -1)^T$, $\boldsymbol{d}' = (3, 0, 1)^T$. Norms: $\sqrt{5}$, $\sqrt{5}$, $\sqrt{10}$ respectively.

6. Part A2 — Rotation order is not interchangeable in 3D

Data: $\boldsymbol{p} = (1, 0, 0)^T$. Two $+90°$ rotations, one around x and one around z, both about **fixed global axes**.

1. First rotate about x, then about z:
$$\boldsymbol{p}_{xz} = \mathbf{R}_z(90°)\mathbf{R}_x(90°)\boldsymbol{p}.$$
2. First rotate about z, then about x:
$$\boldsymbol{p}_{zx} = \mathbf{R}_x(90°)\mathbf{R}_z(90°)\boldsymbol{p}.$$
3. Compare both results with `np.allclose()`. Are they equal? Explain the effect of order.
4. Verify the norms of the initial vector and both transformed vectors.

Expected: $\boldsymbol{p}_{xz} = (0, 1, 0)^T$ and $\boldsymbol{p}_{zx} = (0, 0, 1)^T$; all norms equal 1.

7. Part B — Engineering: 3D force and torque

An actuator force is initially aligned with the global x axis: $\boldsymbol{F}_0 = (10, 0, 0)^T$ N. Its direction is rotated first by $+90°$ about the fixed global z axis and then by $+90°$ about the fixed global x axis.

1. Build $\mathbf{R}_z(90°)$ and $\mathbf{R}_x(90°)$ from sine and cosine.
2. Compute the components after the first turn, then the final force using the ordered product $\boldsymbol{F}_G = \mathbf{R}_x\mathbf{R}_z\boldsymbol{F}_0$.
3. Verify that the force magnitude remains 10 N.
4. The force acts with lever arm $\boldsymbol{r} = (0.2, 0, 0)^T$ m. Compute torque $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}_G$ using `np.cross()` (EX27).
5. Interpret the signs and print force and torque with correct units.

Expected: $\boldsymbol{F}_1 = (0, 10, 0)^T$ N; $\boldsymbol{F}_G = (0, 0, 10)^T$ N; $\boldsymbol{M} = (0, -2, 0)^T$ N m.

8. Part C — Reference-frame change (force remains fixed)

A **fixed** physical force has global components $\boldsymbol{F}_G = (3, 4, 0)^T$ N. The **local axes** turn by $+90°$ around z with respect to the global axes; the force itself **does not move**.

1. Build $\mathbf{R}_z(90°)$.
2. Compute local components using $\boldsymbol{F}_L = \mathbf{R}_z(90°)^T \boldsymbol{F}_G$.
3. Recover global components using $\mathbf{R}_z(90°)\boldsymbol{F}_L$.
4. Compare recovered vectors and norms; explain in one sentence what changed and what did not.

Expected: $\boldsymbol{F}_L = (4, -3, 0)^T$ N, with norm 5 N in both representations.

9. Validation checklist

- Rotations follow the right-hand rule and use degrees converted to radians.
- $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$ are `(3,3)` arrays; vectors are `(3,)` arrays.
- In 3D, $\mathbf{R}_x\mathbf{R}_z \neq \mathbf{R}_z\mathbf{R}_x$ in general.
- Checks cover norm preservation, $\mathbf{R}^T\mathbf{R} \simeq I$, determinant, manual values and recovery.
- Active vector rotation (B) and passive reference-frame change (C) are kept distinct.

10. Python hints: consult at the end

Useful tools (not a complete solution): `np.deg2rad`, `np.sin`, `np.cos`, `np.array`, `@`, `.T`, `np.allclose`, `np.eye`, `np.linalg.norm`, `np.linalg.det` and `np.cross`.

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
c = np.cos(angle_rad)
s = np.sin(angle_rad)
# Build Rx, Ry or Rz using c and s for the chosen axis.
# Verify R.T @ R by comparing with np.eye(3).
```